



Candidate task: Binance WebSocket capture (CSV) + local order book (CSV)

Language: C++ only (recommended **C++17** or newer; compiler and standard must be stated in your README).

Deliver a **single coherent solution**—your own code or extensions to the provided codebase—that:

1. **Connects to Binance** public WebSocket streams, captures **depth**, **depth5**, and **trade** traffic for configured symbols, and writes a **market-data CSV** audit file (one row per inbound event).
2. **Maintains a local order book (LOB)** from that stream **in chronological order**, and after each processed event emits a snapshot row to an **order-book CSV** file using the **exact schema** below.

Binance WebSocket reference (official docs)

Use these as the authoritative stream semantics and URL patterns:

Product	Documentation
Spot — WebSocket market streams (individual & combined)	Spot WebSocket streams
Spot — Combined streams URL shape	Combined streams
USD-M futures — WebSocket market streams	USD-M Futures WebSocket market streams

Example base URLs used in practice for combined streams:

- **Spot:** `wss://stream.binance.com:9443/stream?streams=`
- **USD-M (public book + @trade):** `wss://fstream.binance.com/public/stream?streams=`
(Raw @trade is on the /public path for USD-M; see *Binance derivatives docs*.)

Streams you must subscribe to per symbol (combined stream names, lowercase symbol in URL):

Stream suffix	Role
<code><symbol>@depth@100ms</code>	Differential depth (<code>depthUpdate</code>)
<code><symbol>@depth5@100ms</code>	Partial top-of-book snapshot (sanity check / refresh)
<code><symbol>@trade</code>	Individual trades

Example combined path fragment for one symbol (spot):

```
btcusdt@depth@100ms/btcusdt@depth5@100ms/btcusdt@trade
```

Deliverable A — Market data dump (*.csv)

One row per message your process applies from the socket (after parsing the envelope so you store the logical Binance data payload).

File naming

- `market_data_<venue>_<SYMBOL>_<UTC-date>.csv` , or one file per run with a `symbol` column if multi-symbol — **document your choice**.

CSV rules

- UTF-8, **RFC 4180** style: comma-separated; fields containing comma, quote, or newline **double-quoted**; internal " escaped as `""` .
- **First row = header** with the exact column names below (order fixed).



Required columns

Column	Description
recv_tsec	int64 — seconds part of receive time (see time policy)
recv_tnsec	int32 — nanosecond remainder in [0, 999_999_999]
venue	spot or usdm
stream_kind	depth_diff depth5 trade (or 0 / 1 / 2 if documented)
shard_id	Non-negative integer shard index (0 if single connection)
conn_epoch	Non-negative integer; increment on each WebSocket reconnect
conn_seq	Monotonic within (shard_id, conn_epoch)
symbol	Uppercase, e.g. BTCUSDT
payload_json	The inner Binance data object as minified JSON (one line), CSV-escaped

Time policy (pick one, document):

- **Recommended:** wall-clock when your process finished reading the message (e.g. split `recv_ns` into `recv_tsec` / `recv_tnsec` if you use nanoseconds).
- **Alternative:** Binance event time from JSON — document field (`E` , `T` , ...) and units.

Ordering: Rows must reflect **processing order** (same order your LOB consumer would see).

Deliverable B — Order book snapshots (`*_orderbook.csv`)

For each symbol you support, emit **one CSV row after each event** that changes or refreshes the book state you model (at minimum: every applied `depthUpdate` and `depth5` ; **trades** optional but if included, document how they affect or only annotate the row).

CSV rules

- Same as above: UTF-8, commas, quoted fields when needed.
- **First row = header** — must be **exactly** this header (single line):

```
tsec,tnsec,seqNo,id,type,side,bid0,bid1,bid2,bid3,bid4,bid_size0,bid_size1,bid_size2,bid_size3,bid_size4,ask0,ask1,ask2,ask3,ask4,ask_size0,ask_size1,ask_size2,ask_size3,ask_size4
```

Column semantics (data types and meaning)

Column	Type	Meaning
tsec	int64	Integer seconds for the row timestamp
tnsec	int32	Nanosecond remainder in [0, 999_999_999] ; <code>tsec * 1e9 + tnsec</code> = event time in ns
seqNo	uint64	Monotonic application sequence (one increment per emitted row)
id	int32	Stable numeric instrument id for the session (document how derived)
type	char	Single ASCII letter for last applied event class (document mapping)
side	char	<code>B</code> = bid-side, <code>S</code> = ask-side, <code>N</code> = symmetric / N/A
bid0 ... bid4	int64	Top 5 bid prices , best first, scaled integers
bid_size0 ... bid_size4	int64	Sizes at those bids, same scaling
ask0 ... ask4	int64	Top 5 ask prices , best first, scaled integers
ask_size0 ... ask_size4	int64	Sizes at those asks, same scaling

Timestamp policy

- **Recommended:** derive from the same source as the market dump row for that event (e.g. `recv_tsec` / `recv_tnsec`), **or** from Binance — document clearly.

Integer scaling (mandatory documentation)

Binance sends decimals as **strings**. You must document fixed **price** and **quantity** scales (e.g. 10^8 ticks) and apply them **deterministically** (no binary floats in the CSV integer columns).

Reconnects and gaps

- Document how `conn_epoch` (and Binance `U` / `u` / `pu` on diffs) affects your LOB: reset, resync, or drop until snapshot.

What to submit

1. **C++ source code** (no alternative runtime languages), **CMake** (or justification + reproducible build for a common Linux toolchain), and **README** with:
 - compiler, **C++ standard**, dependencies, full build commands;
 - exact CLI to produce both CSV outputs;
 - symbol list format;
 - scaling and timestamp policies;
 - reconnect / gap handling summary;
 - brief **design note** on threading and I/O (see review focus below).
2. **Short sample run** (e.g. 1–2 minutes, one symbol) with:
 - `market_data_*.csv` (truncated in README or attached if small);
 - `*_orderbook.csv` corresponding to the same window.
3. **No network calls** during “grading replay” if you provide a **replay mode** that reads your own `market_data_*.csv` and reproduces the order book — state this in the README.

Acceptance checklist (functional)

#	Criterion
1	Live capture from Binance combined streams with <code>depth@100ms</code> , <code>depth5@100ms</code> , <code>trade</code>
2	Market dump CSV matches column contract and row-per-event ordering
3	Order book CSV header matches exactly ; 26 data columns per row after header
4	Top-5 bid/ask reflect correct diff semantics (qty <code>0</code> removes level)
5	Integer scaling and timestamps documented; output reproducible from fixed input
6	Reconnect / sequence / epoch behavior stated

What reviewers will check (code review & performance)

Candidates should **design and optimize for** the items below. Reviewers will look for evidence in code, build flags, and README—not just “it runs.”

Correctness & market-data literacy

- **Stream semantics:** Differential depth vs `depth5` replace semantics; trade stream does not automatically update a diff-based book unless you document integration.
- **Sequence / gaps:** Handling of `U` / `u` / `pu`, and behavior on new `conn_epoch` after reconnect (documented and consistent with CSV output).



- **Determinism:** Integer scaling for prices/sizes; **no double in the hot path to CSV integers** unless justified and lossless; stable ordering of levels (best bid/ask first).
- **CSV contract:** RFC 4180 escaping for `payload_json` ; no silent truncation; header rows exact.

C++ quality & maintainability

- **Ownership & lifetimes:** Clear RAI; no dangling references across async callbacks; websocket + I/O teardown on shutdown (`SIGINT` / `SIGTERM`) without leaks (Valgrind/sanitizer friendly).
- **Error handling:** Reconnect paths, partial reads, parse failures—logged or counted, not silent UB.
- **Modern C++:** Prefer `constexpr` / `enum class` /scoped types where it helps; avoid unnecessary macros; **const correctness** on hot structures.
- **Build hygiene:** `-Wall -Wextra` (or stricter), `Release` build documented; no warnings left unexplained.

Performance & scalability (prioritize these)

Reviewers often probe **where CPU and allocations go**:

- **JSON parsing:** Efficient parser choice (e.g. simd-style or documented fallback); avoid re-parsing the same payload unnecessarily; reuse buffers where safe.
- **Allocations:** Hot path should **minimize** `std::string` churn and per-message heap hits; pooling, `reserve` , ring buffers, or SBO-friendly patterns score well if used correctly.
- **Order book updates:** Updating top **5** levels should be **O(updates)** per message with appropriate structure (candidate explains trade-offs vs full-map book).
- **I/O:** File/socket back-pressure considered; optional **buffered writes** or dedicated writer thread vs blocking the network thread—document choice and why.
- **Threading:** If multi-threaded, clear boundaries: no data races (document synchronization); avoid contended locks on every tick if you claim high throughput.

Observability & reviewability

- **Metrics:** Counters for messages processed, parse errors, reconnects, rows written—helps reviewers validate robustness.
- **Replay mode:** Ability to regenerate `*_orderbook.csv` from saved `market_data_*.csv` without network makes review and regression much easier.

Security & ops (baseline)

- No secrets in repo; TLS verification not disabled without strong justification.
- Reasonable limits (max URL/shard size) to avoid runaway memory if misconfigured.

Optional stretch

- REST depth snapshot + diff buffer resync per Binance documentation.
- Multi-symbol with correct sharding.
- Performance notes (throughput, latency).

Here is the **GitHub submission section** to add at the end of your README:

How to Submit on GitHub

1. Initialize and push your repository

```
# From your project root
git init
git add .
git commit -m "Initial submission: Binance WebSocket capture + LOB"
```

```
# Create a new repo on GitHub (via web UI or CLI), then:
git remote add origin https://github.com/<your-username>/binance-lob-capture.git
git branch -M main
git push -u origin main
```

2. Make sure `.gitignore` excludes build artifacts and secrets

```
# .gitignore
build/
*.csv
*.o
*.a
*.so
.env
config/secrets.json
```

Never commit API keys, private keys, or credentials. All config should use environment variables or a `.env.example` template (no real values).

3. Tag your submission

```
git tag -a v1.0.0 -m "Submission v1.0.0"
git push origin v1.0.0
```

Full Build & Run Commands (Reproducible)

Prerequisites

```
# Ubuntu 22.04 / Debian — install all dependencies
sudo apt-get update
sudo apt-get install -y \
  cmake ninja-build \
  g++-12 \
  libssl-dev \
  libboost-all-dev \
  zlib1g-dev
```

Compiler: GCC 12 (`g++-12`), Standard: C++17. Also tested with Clang 15.

Clone and build

```
git clone https://github.com/<your-username>/binance-lob-capture.git
cd binance-lob-capture

cmake -B build -G Ninja \
  -DCMAKE_BUILD_TYPE=Release \
  -DCMAKE_CXX_COMPILER=g++-12 \
  -DCMAKE_CXX_FLAGS="-Wall -Wextra -O2"

cmake --build build --parallel
```

Run — Live capture (both CSVs)

```
# Single symbol, spot venue, 2-minute capture
./build/binance_capture \
--venue spot \
--symbols BTCUSDT \
--output-dir ./output
```

```
# Multi-symbol
./build/binance_capture \
--venue spot \
--symbols BTCUSDT,ETHUSDT \
--output-dir ./output
```

```
# USD-M futures
./build/binance_capture \
--venue usdm \
--symbols BTCUSDT \
--output-dir ./output
```

Outputs written to `./output/` :

```
output/
market_data_spot_BTCUSDT_2025-01-15.csv
market_data_spot_BTCUSDT_2025-01-15_orderbook.csv
```

Run — Verify CSV schema

```
# Check header and row count
head -2 ./output/market_data_spot_BTCUSDT_2025-01-15.csv
head -2 ./output/market_data_spot_BTCUSDT_2025-01-15_orderbook.csv

# Confirm orderbook has exactly 26 data columns (25 commas per data row)
awk -F',' 'NR==2{print NF}' ./output/market_data_spot_BTCUSDT_2025-01-15_orderbook.csv
# Expected output: 26
```

Run tests (if applicable)

```
cmake --build build --target tests
./build/tests/unit_tests
```

Reviewer Checklist

Step	Command
Build from clean	<code>cmake -B build && cmake --build build</code>
Live 300s capture	<code>./build/binance_capture --venue spot --symbols BTCUSDT --duration 300</code>
Column count check	<code>awk -F',' 'NR==2{print NF}' *_orderbook.csv</code> 26
No secrets in repo	<code>git grep -i "api_key secret password"</code> empty